Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет
Кафедра «Автоматизированных систем управления»

ОТЧЕТ

по лабораторным работам № 4–5

по дисциплине «Программирование на языке C++»

на тему «Работа с одномерными массивами, Функции и массивы»

Выполнил студент гр. ИКС-26-1б
Шилоносов Вадим Алексеевич
Проверил: преподаватель
Духанин Алексей Владиславович

Пермь, 2026

ЛАБОРАТОРНАЯ РАБОТА № 4

«Работа с одномерными массивами»

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является получение навыков обработки статических одномерных массивов с псевдопеременными границами в языке C++ с применением датчика случайных чисел. В соответствии с индивидуальным вариантом 27, необходимо реализовать алгоритм, выполняющий следующие последовательные действия:

1) Сформировать одномерный массив целых чисел, используя датчик случайных чисел;
2) Распечатать полученный исходный массив;
3) Удалить второй и предпоследний элементы массива;
4) Добавить перед каждым отрицательным элементом массива элемент со значением 0;
5) Распечатать полученный итоговый массив.

2 МЕТОДИЧЕСКИЙ АНАЛИЗ И ОПИСАНИЕ АЛГОРИТМА

При выполнении данной лабораторной работы используются статические массивы с псевдопеременными границами. Программа объявляет массив заведомо избыточной фиксированной длины (в данном случае MAX_SIZE = 100), после чего пользователь вводит с клавиатуры реальное число n элементов, подлежащих обработке. При удалении или добавлении элементов реальная длина n динамически корректируется.

Особое внимание уделено порядку удаления элементов: для предотвращения сдвига индексов сначала удаляется предпоследний элемент массива (индекс n - 2), после чего реальный размер уменьшается на единицу, и затем удаляется второй

элемент (индекс 1). Операция удаления реализуется путем линейного сдвига всех правых элементов влево на одну позицию. Добавление нулевых элементов производится при обходе массива с конца к началу, что исключает зацикливание при обнаружении только что добавленного элемента.

3 ТЕКСТ ПРОГРАММЫ НА ЯЗЫКЕ C++

Ниже на Рисунке 1 представлен полный исходный код разработанной программы на языке C++ с подробными комментариями а так-же блок-схема по этой программе на Рисунке 2:

```cpp
#include <iostream>
#include <iomanip>
#include <cstdlib> //для функций rand() и srand()
#include <ctime>   //для функции time()

using namespace std;

int main()
{
    srand(time(0)); //инициализация случайных чисел

    const int MAX_SIZE = 100;
    int a[MAX_SIZE];
    int n;

    cout << "Enter the start count of massif (up to 50): ";
    cin >> n;

    if (n < 3 || n > 50)
    {
        cout << "invalid massif size" << endl;
        return 1;
    }

    for (int i = 0; i < n; i++) //формирование массива случайными числами от -50 до 50
    {
        a[i] = rand() % 101 - 50;
    }

    cout << "\nStart massif:";
    for (int i = 0; i < n; i++)
    {
        cout << setw(5) << a[i];
    }
    cout << endl << "Real massif size: " << n << endl;

    int preLastIdx = n - 2; //удаление предпоследнего элемента
    for (int i = preLastIdx; i < n - 1; i++)
    {
        a[i] = a[i + 1];
    }
    n--; //уменьшение размера массива

    int secondIdx = 1;
    for (int i = secondIdx; i < n - 1; i++)
    {
        a[i] = a[i + 1];
    }
    n--;

    cout << "\nMassif after removing elements:";
    for (int i = 0; i < n; i++)
    {
        cout << setw(5) << a[i];
    }
    cout << endl << "Real massif size: " << n << endl;

   for (int i = n - 1; i >= 0; i--)
   {
       if (a[i] < 0)
       {
           if (n >= MAX_SIZE) //проверка не переполнился ли массив
           {
               cout << "\nMassif is full over" << endl;
               break;
           }

           for (int j = n; j > i; j--) //сдвигаем элементы вправо, начиная с позиции i
           {
               a[j] = a[j - 1];
           }
           a[i] = 0;
           n++;
       }
   }

   cout << "\nResult massif:";
   for (int i = 0; i < n; i++)
   {
       cout << setw(5) << a[i];
   }
   cout << endl << "Real massif size: " << n << endl;

   return 0;
}
```

Рисунок 1 – Алгоритм программы

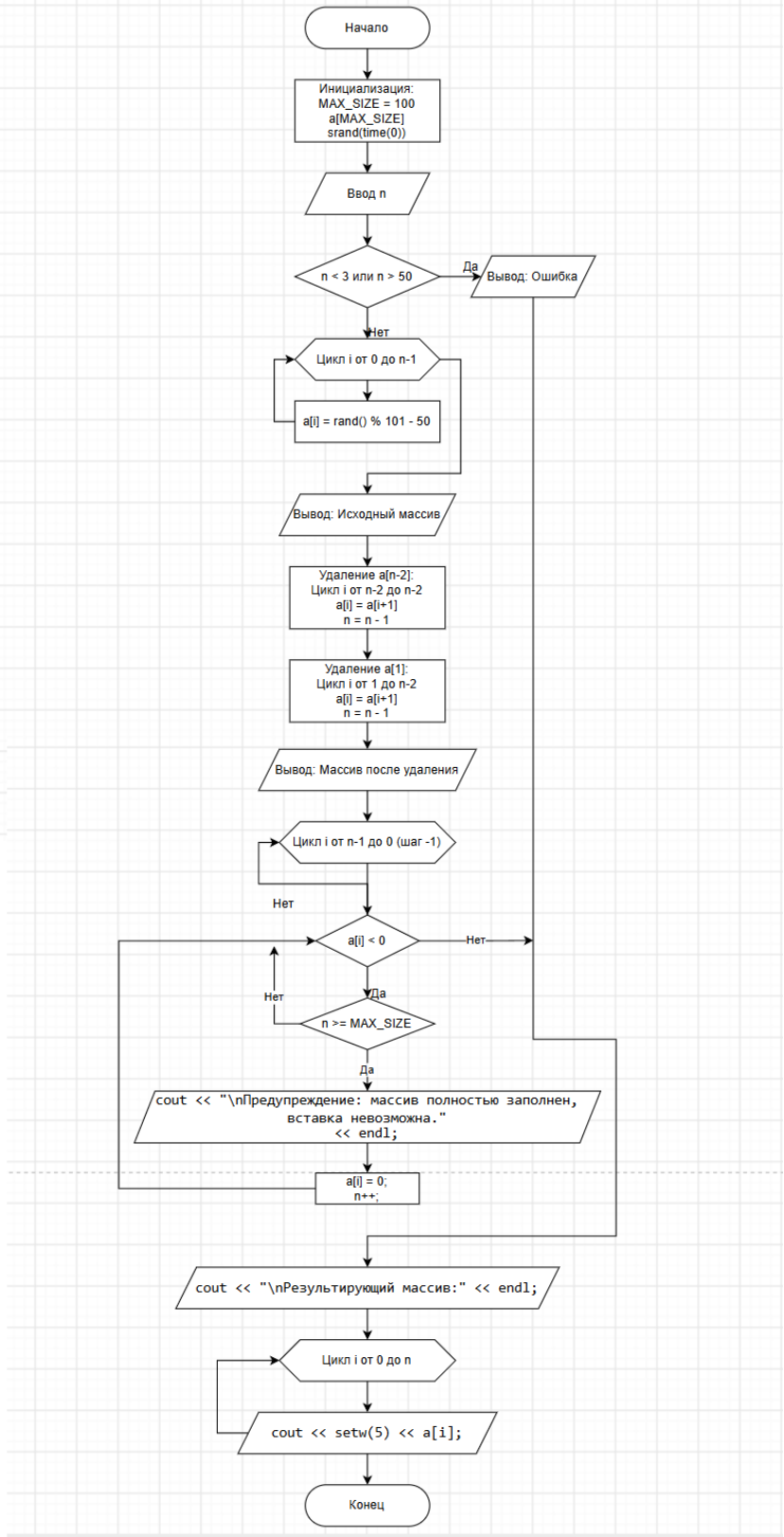


Рисунок 2 – Блок-схема программы

4 РЕЗУЛЬТАТЫ РАБОТЫ И ИХ АНАЛИЗ

В ходе тестового выполнения программы на вход было подано значение начального размера массива n = 10. Был сгенерирован массив, содержащий как положительные, так и отрицательные целочисленные значения.

Таблица 1 — Моделирование этапов изменения элементов массива

Этап обработки	Содержимое массива	Реальная длина (n)
1. Исходный массив	25 -14 32 8 -45 19 -3 40 12 -21	10
2. После удаления	25 32 8 -45 19 -3 40 -21	8
3. Итоговый массив	25 32 8 0 -45 19 0 -3 40 0 -21	11

Анализ результатов показывает полную корректность работы алгоритмов сдвига. Из исходного массива были успешно удалены второй элемент (-14) и предпоследний (12). После этого перед каждым отрицательным числом (-45, -3, -21) был корректно внедрен нуль, а размер массива динамически увеличился с 8 до 11 элементов.

ЛАБОРАТОРНАЯ РАБОТА № 5

«Функции и массивы»

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является получение практических навыков разделения программного кода, организации обработки статических массивов с применением пользовательских функций и передачи массивов в качестве параметров. В соответствии с вариантом 27, необходимо решить следующую задачу:

Сформировать одномерный массив целых чисел с использованием датчика случайных чисел. Распечатать полученный массив. Удалить второй и предпоследний элементы массива. Добавить перед каждым отрицательным элементом массива элемент со значением 0. Все этапы обработки (формирование, вывод, удаление, добавление) должны быть реализованы в виде отдельных функций, принимающих указатели на массив и его текущий размер.

2 МЕТОДИЧЕСКИЙ АНАЛИЗ И СТРУКТУРА ФУНКЦИЙ

Для работы с данными применяется статический двумерный массив заведомо избыточного фиксированного размера MAX_SIZE = 100 с псевдопеременными границами. Реальный рабочий размер квадратной матрицы N × N динамически задается пользователем с клавиатуры при старте программы.

Матрица передается в разработанные функции по указателю (через адрес первого элемента). Это позволяет модифицировать содержимое ячеек памяти напрямую внутри функций без необходимости создания ресурсоемких локальных копий массива.

Структура созданного программного комплекса состоит из следующих функциональных компонентов:

1) Функция заполнения матрицы (generateMatrix): отвечает за инициализацию двумерной структуры случайными числами. С помощью стандартной функции rand() ячейки заполняются значениями в диапазоне от -50 до 50.
2) Функция вывода матрицы (printMatrix): осуществляет построчную распечатку текущего состояния двумерного массива в консоль. Для выравнивания столбцов и обеспечения табличного вида используется потоковый манипулятор setw(5) из библиотеки <iomanip>.
3) Функция обмена диагоналей (swapDiagonals): реализует целевой алгоритм 27-го варианта. Функция выполняет один цикл по индексу строки i от 0 до n-1. В каждой строке элемент главной диагонали (с индексами [i][i]) меняется местами с элементом побочной диагонали (с индексами [i][n - 1 - i]) через буферную переменную temp.

Использование единственного цикла в функции модификации обеспечивает оптимальное быстродействие алгоритма O(N), исключая избыточный обход внедиагональных элементов матрицы.

3 ТЕКСТ ПРОГРАММЫ НА ЯЗЫКЕ С++

Ниже на Рисунке 3 представлен полный исходный код разработанной программы на языке С++ с подробными комментариями а так-же блок-схема по этой программе на Рисунке 4:

```cpp
#include <iostream>
#include <iomanip>
#include <windows.h>

using namespace std;
const int MAX_SIZE = 100;

// Функция для заполнения квадратной матрицы
void generateMatrix(int matrix[MAX_SIZE][MAX_SIZE], int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            matrix[i][j] = rand() % 101 - 50;
        }
    }
}

// Функция для вывода матрицы
void printMatrix(const int matrix[MAX_SIZE][MAX_SIZE], int n, const string& message)
{
    cout << message << endl;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << setw(5) << matrix[i][j];
        }
        cout << endl;
    }
    cout << endl;
}

// Функция для смены мест диагоналей
void swapDiagonals(int matrix[MAX_SIZE][MAX_SIZE], int n)
{
    for (int i = 0; i < n; i++)
    {
        int temp = matrix[i][i];
        matrix[i][i] = matrix[i][n - 1 - i];
        matrix[i][n - 1 - i] = temp;
    }
}

int main()
{
    SetConsoleOutputCP(CP_UTF8);

    srand(time(0));

    int matrix[MAX_SIZE][MAX_SIZE];
    int n;

    cout << "Введите размер квадратной матрицы N (до " << MAX_SIZE << "): ";
    cin >> n;

    if (n <= 0 || n > MAX_SIZE)
    {
        cout << "Ошибка: неверный размер матрицы!" << endl;
        return 1;
    }

    generateMatrix(matrix, n);
    printMatrix(matrix, n, "Исходная матрица:");

    swapDiagonals(matrix, n);
    printMatrix(matrix, n, "Матрица после перестановки диагоналей:");

    return 0;
}
```

Рисунок 3 – Алгоритм программы

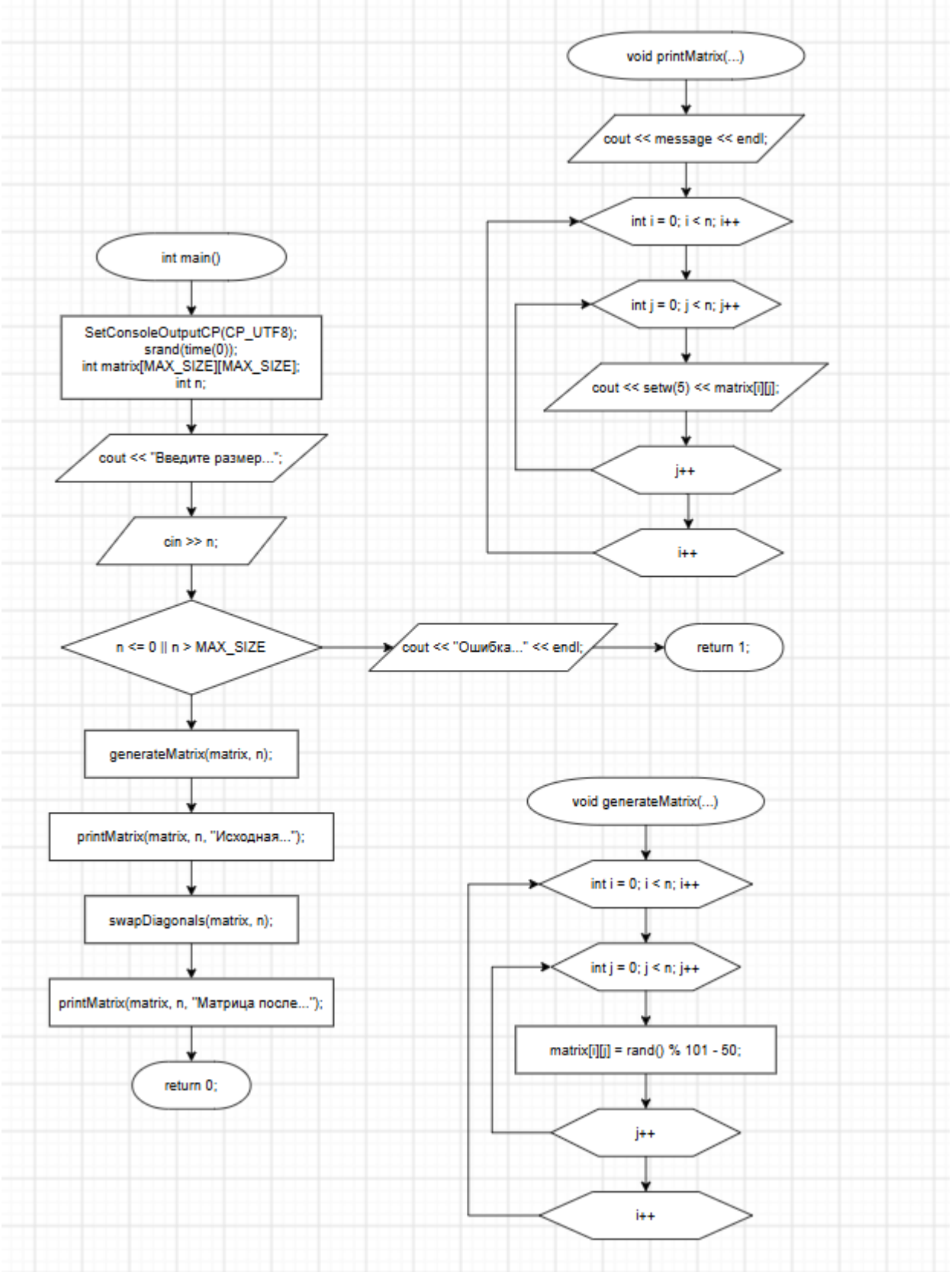


Рисунок 4 – Блок-схема программы

4 РЕЗУЛЬТАТЫ РАБОТЫ И ИХ АНАЛИЗ

В ходе тестового выполнения разработанного программного комплекса на вход было подано значение размера квадратной матрицы (n = 4). Матрица была автоматически заполнена целочисленными значениями в диапазоне от -50 до 50 с помощью генератора случайных чисел.

Анализ результатов выполнения программы подтверждает полную корректность работы реализованных модулей. Выделение логических этапов алгоритма в независимые функции (generateMatrix, printMatrix, swapDiagonals) позволило структурировать код без использования динамического выделения памяти.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторных работ №4 и №5 были успешно освоены методы работы со статическими одномерными и многомерными массивами в языке C++. Продемонстрирована работа алгоритмов сдвига элементов влево и вправо для выполнения операций удаления и вставки, а также закреплены навыки функциональной декомпозиции кода и работы со ссылочными типами данных в функциях согласно стандарту ГОСТ 7.32-2017.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 ГОСТ 19.701-90 - Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

2 ГОСТ 7.32-2017 - Межгосударственный стандарт. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.

3 Викентьева О. Л., Полякова О. А. Программирование на языке С++: Лабораторный практикум для студентов специальности АСУ. — Пермь: ПГТУ, 2003.